

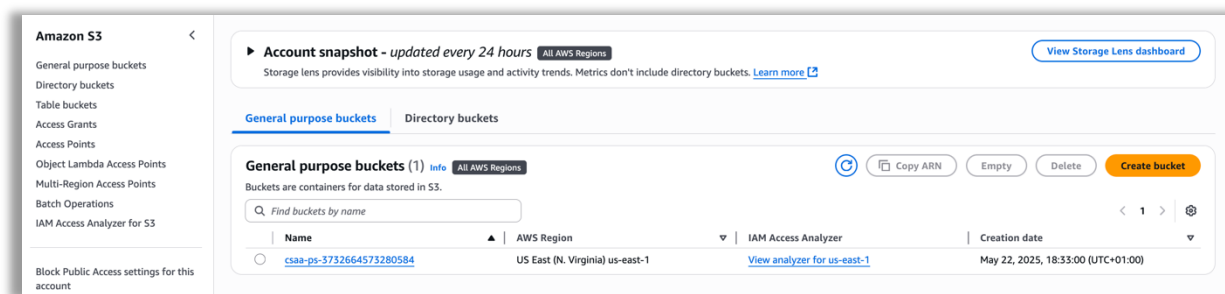
Simulating Recovery of Deleted Files in S3 Using Versioning

Introduction – Christopher Samugana

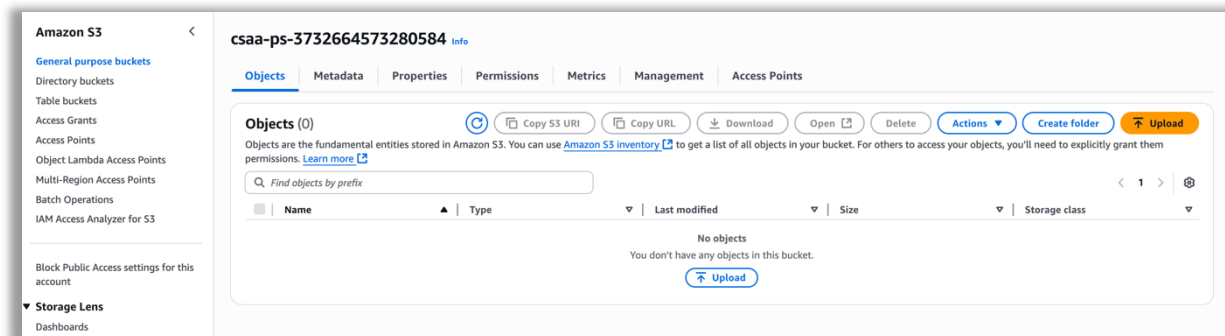
In this lab, I implemented an architecture to mitigate the risk of accidental or intentional deletion of important files stored in Amazon S3. This was achieved using S3 Versioning, which enables recovery of deleted files.

1. Created Test Files

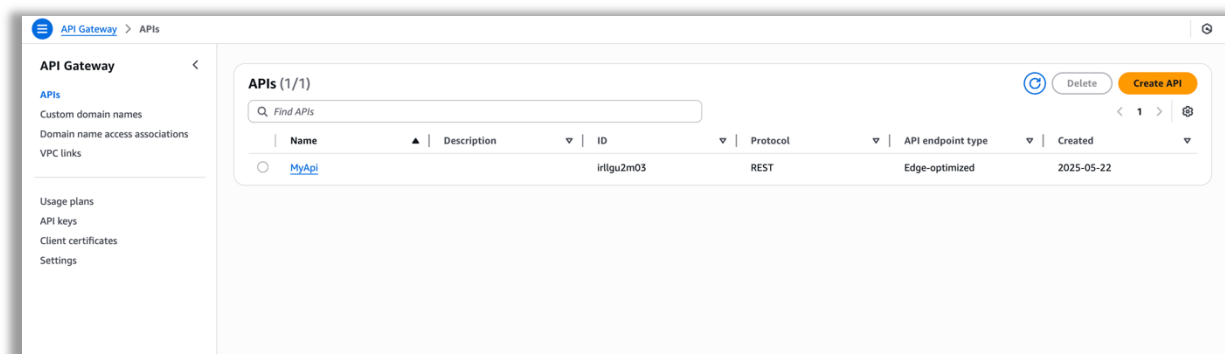
I began by logging into the AWS Management Console and verified that the us-east-1 region was selected.



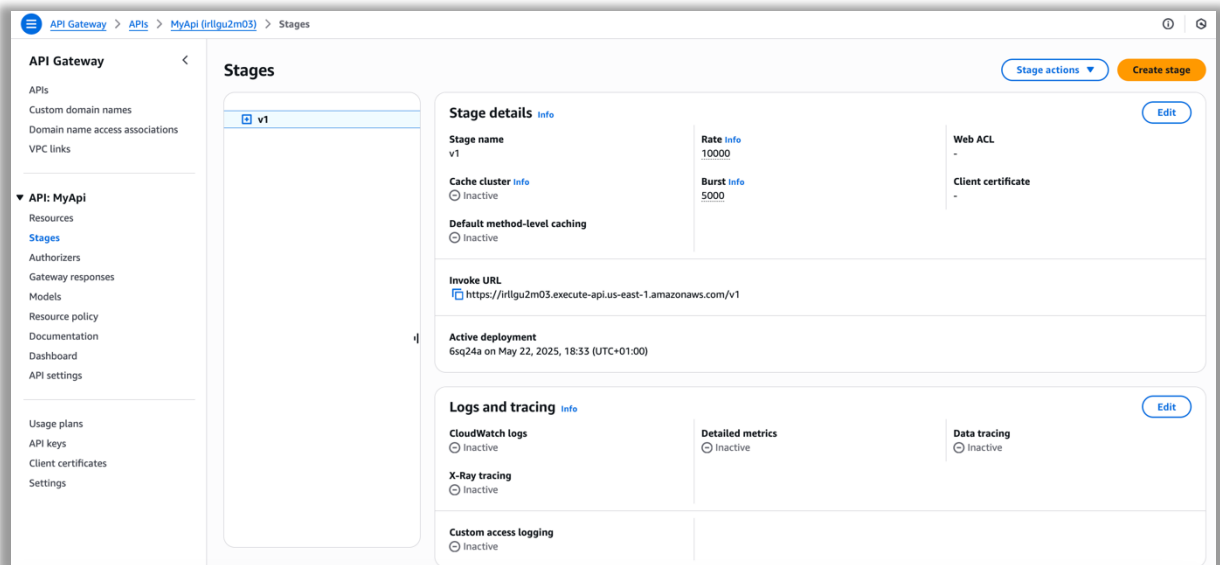
From the Amazon S3 dashboard, I located the pre-provisioned S3 bucket, which began with the prefix csaa-ps-. At this point, the bucket was empty.



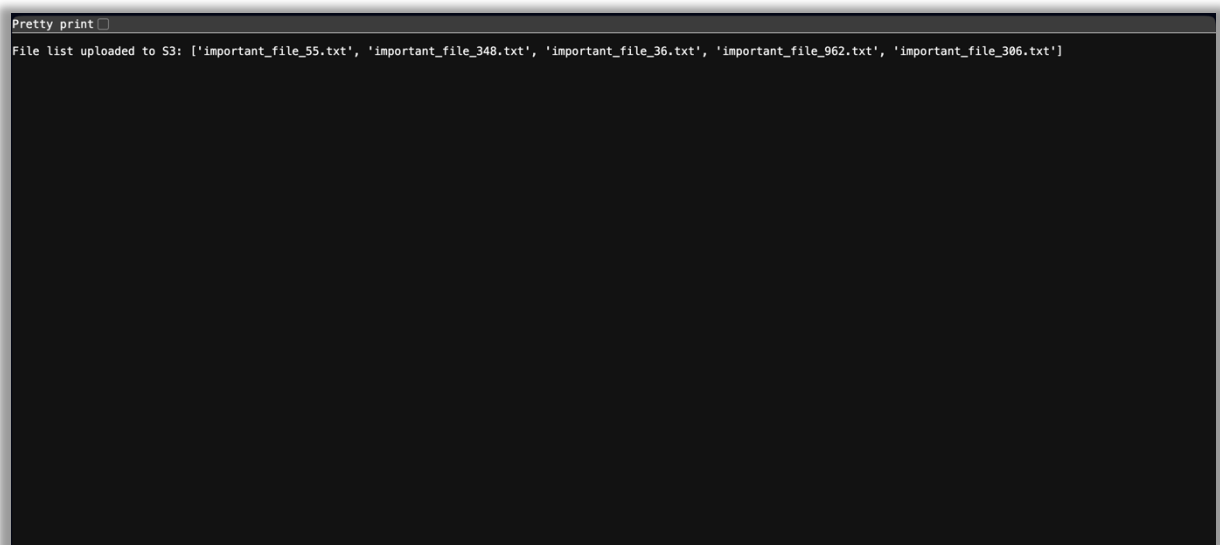
To initiate the creation of test files, I opened the Amazon API Gateway console in a new tab and selected the MyAPI REST API from the list.



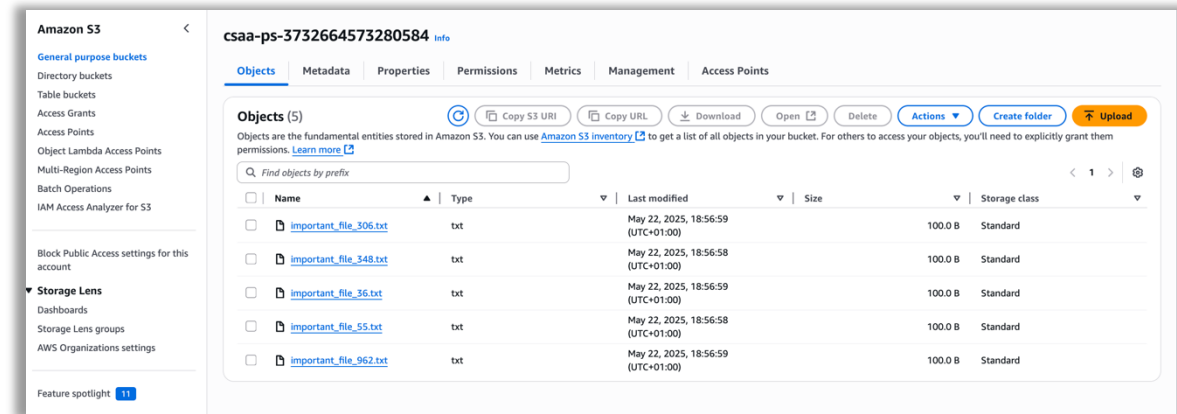
Within the Stages section under the MyAPI API, I selected the v1 stage and copied the Invoke URL.



Opening this URL in a browser tab triggered a response confirming the creation of test files in the S3 bucket. The response returned a list of uploaded files:

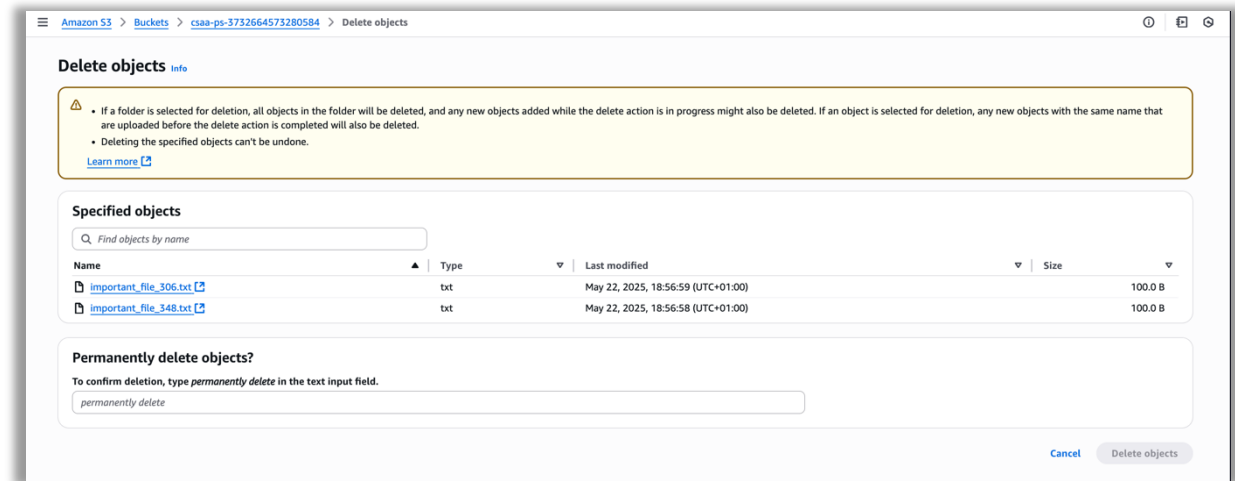


I returned to the S3 bucket and refreshed the objects list to confirm the files were present.

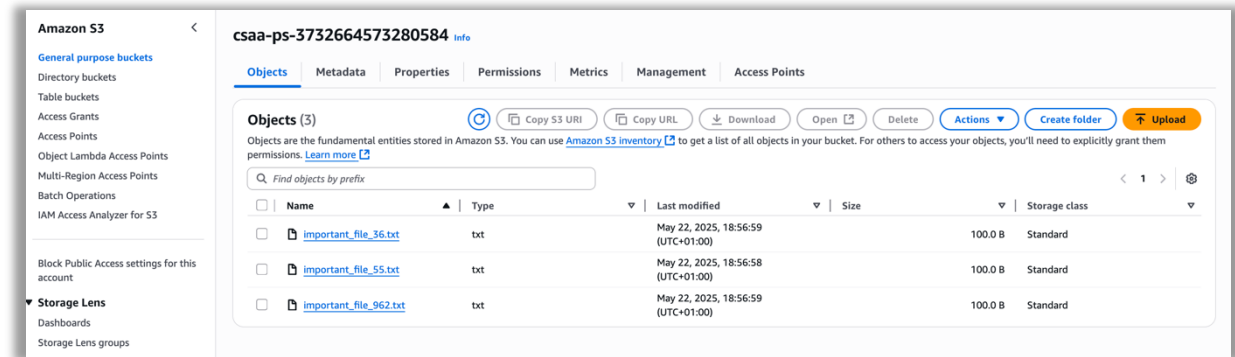


2. Deleted Objects (Pre-Versioning)

Next, I selected one or two of the `important_file_XXX.txt` objects in the bucket and clicked Delete.



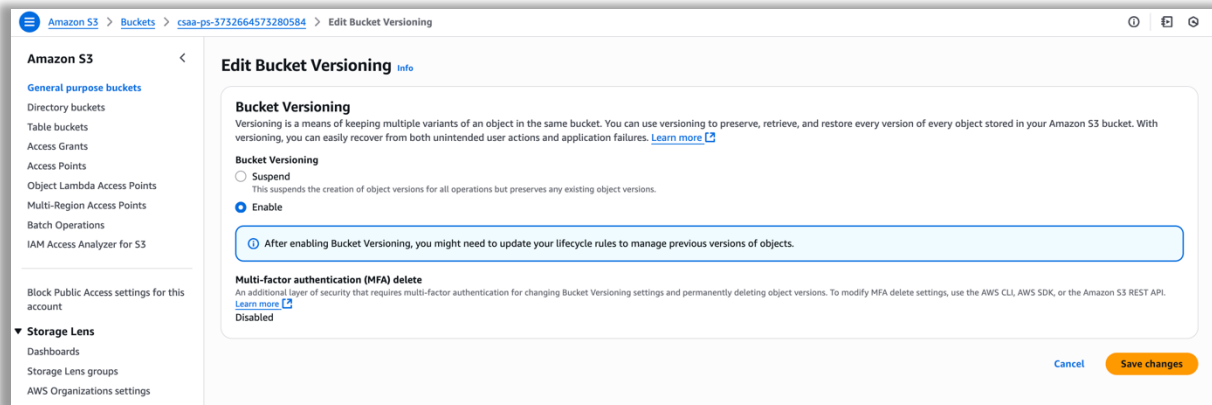
After confirming the prompt to permanently delete the objects, I verified that the selected files were no longer available and could not be recovered. This illustrated the risk of permanent data loss without versioning enabled.



3. Enabled Versioning for the S3 Bucket

To enable file recovery, I navigated to the Properties tab of the S3 bucket.

Under the Bucket Versioning section, I clicked Edit, selected Enable, and saved the changes. Versioning was now successfully enabled on the bucket.

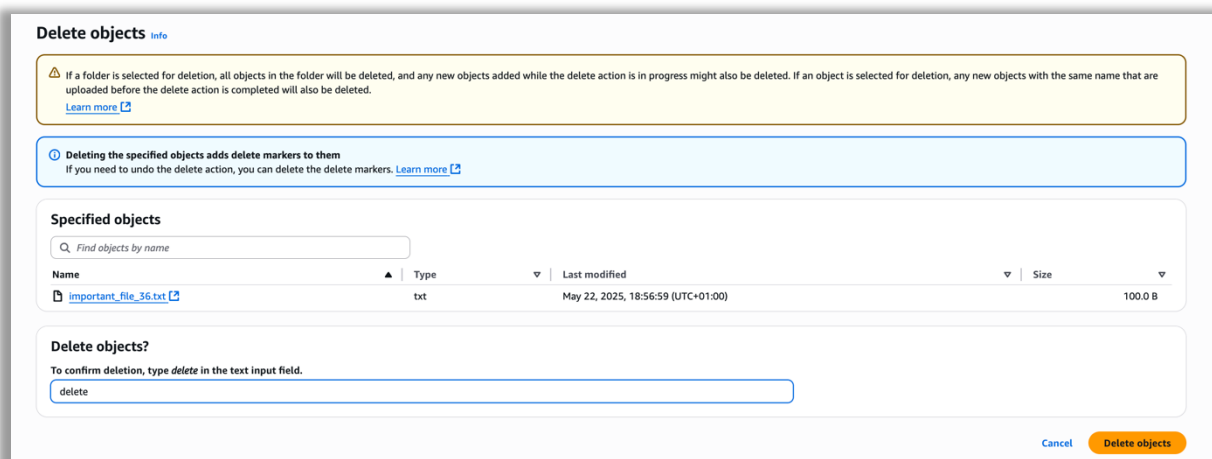
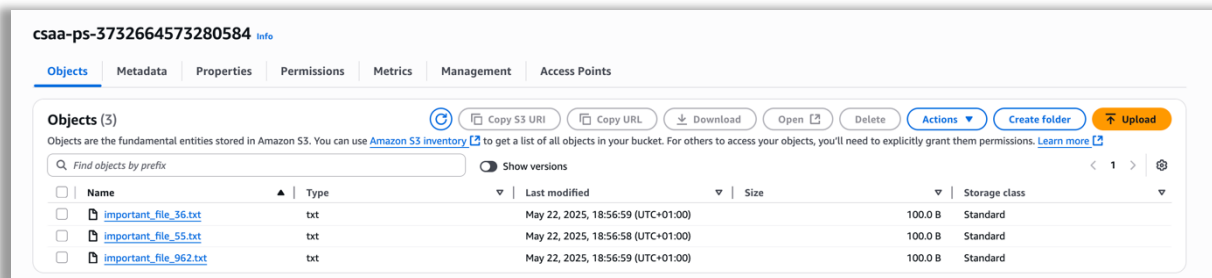


4. Deleted Objects (With Versioning Enabled)

I returned to the Objects tab of the S3 bucket.

I selected a different file for deletion and made a note of its file name. I then deleted the file using the Delete option.

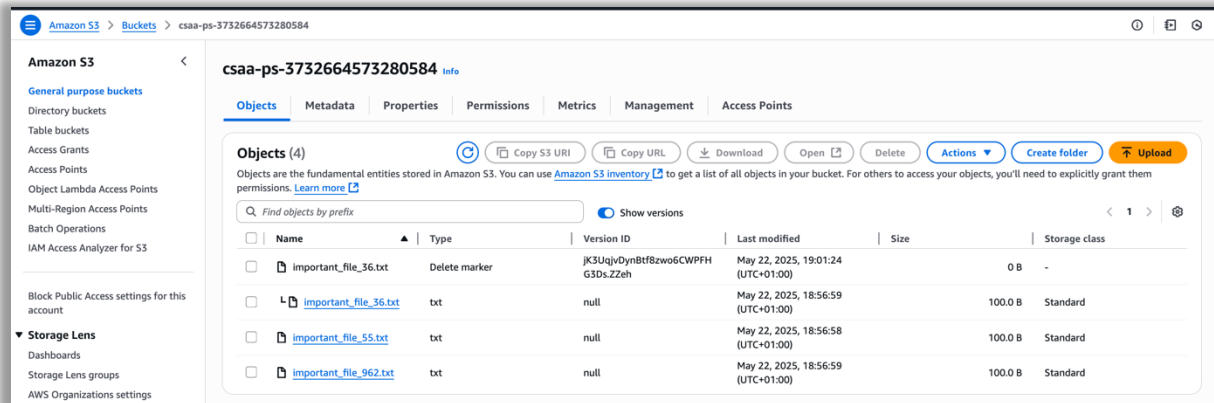
This time, the deletion confirmation prompt indicated a soft delete, not a permanent one.



After deletion, although the object no longer appeared in the default view, it had not been permanently removed thanks to versioning.

5. Recovered a Deleted Object

To recover the deleted file, I clicked the Show versions toggle at the top of the object list. This revealed multiple versions of each object, including a Delete marker for the file that had just been deleted.



I selected the Delete marker entry and permanently deleted it.

Delete objects info

⚠️ If a folder is selected for deletion, all objects in the folder will be deleted, and any new objects added while the delete action is in progress might also be deleted. If an object is selected for deletion, any new objects with the same name that are uploaded before the delete action is completed will also be deleted.

- Deleting the specified objects can't be undone.

[Learn more](#)

Specified objects

Find objects by name

Name	Version ID	Type	Last modified	Size
important_file_36.txt	jK3UqIvDynBtf8zwo6CWPFFH63Ds.ZZeh	Delete marker	May 22, 2025, 19:01:24 (UTC+01:00)	0 B

Permanently delete objects?

To confirm deletion, type *permanently delete* in the text input field.

permanently delete

[Cancel](#) [Delete objects](#)

Upon doing so, the original file reappeared in the object list—successfully restored without re-uploading it.

Conclusion

By enabling Amazon S3 Versioning, I was able to simulate the recovery of deleted files. The first set of deletions, done before versioning, resulted in permanent data loss. However, after enabling versioning, I demonstrated how to restore a deleted file by removing its delete marker. This showcases a practical method for protecting critical data stored in S3 against accidental or malicious deletion.